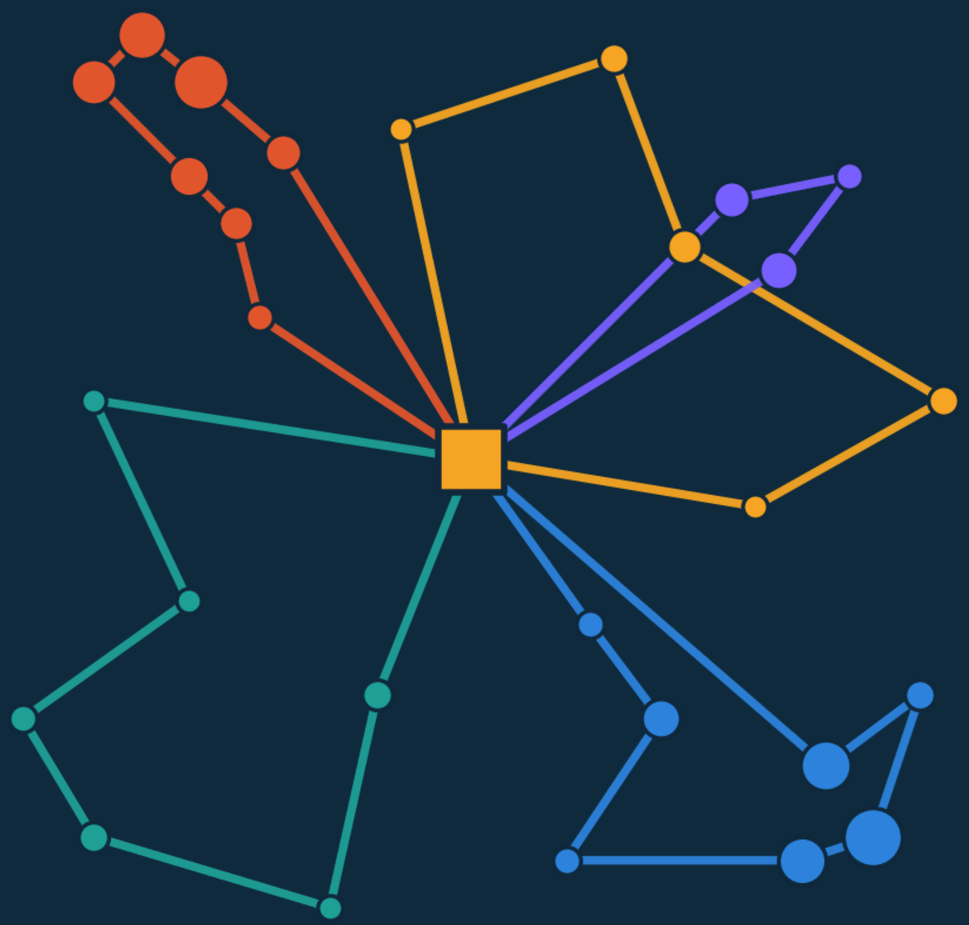
for Green Logistic

AI Hackathon Route Optimization 2026

จาก CVRP Benchmark สู่ First-Mile Seller Pickup



Google OR-Tools · Python

Core Dilemma: ระยะทางสั้นที่สุด = เหมาะสมที่สุด?

“ ในโลกธุรกิจจริง...เส้นทางที่สั้นที่สุดบนแผนที่ จะช่วยบริษัทประหยัดเงินได้จริงหรือเปล่า? ”

Normal VRP

- เป้าหมาย: มุ่งเน้นลดปัญหาระยะทางเป็นหลัก
- รถขนส่ง: เหมือนกันหมด ต้นทุนเท่ากัน
- ร้านค้า: เปิดรับของตลอด 24 ชั่วโมง
- ผลลัพธ์: ได้เส้นทางที่สั้นที่สุดในแผนที่

Business Reality

- เป้าหมาย: ต้นทุนรวม (Total Cost) ไม่ใช่แค่ระยะทาง
- รถขนส่ง: หลากหลายขนาด เช่น รถใหญ่มีค่าบำรุง ค่าน้ำมัน ค่าเช่า ที่มากกว่ารถเล็ก
- ร้านค้า: ระยะเวลาในการยกของขึ้น-ลง
- ผลลัพธ์: การวิ่งเส้นทางที่สั้นแต่ใช้รถผิดประเภท อาจมีต้นทุนสูงกว่าการวิ่งอ้อม

3 ความท้าทายหลักที่ทำให้โจทย์ VRP โลกจริงซับซ้อน

01

Heterogeneous Fleet
รถหลากขนาด & ต้นทุนไม่เท่ากัน

ในกองรถที่มีทั้งรถเล็กประหยัดพลังงาน (ต้นทุนต่ำ) และรถใหญ่บรรทุกหนัก (ต้นทุนสูง)

ความท้าทาย: AI ต้องตัดสินใจ 2 ชั้น คือ 'เลือกรถให้ถูกคัน' ควบคู่กับ 'จัดเส้นทางให้คุ้มค่า'

02

Strict Time Windows
กรอบเวลาเข้ารับ-ส่งของที่ตายตัว

ร้านค้าเปิด-ปิดเป็นรอบเวลา เช่น 9:00 - 11:30 น. และมีเวลาจอดลงของ (Service Time) หน้างาน

ความท้าทาย: ต่อให้เส้นทางสั้นแค่ไหน ถ้าไปถึงตอนร้านปิด แผนจะกลายเป็น Infeasible ทันที

03

Operation Limits
ชั่วโมงทำงานคนขับ & สินค้าสด

เวลารวมทั้งกะจำกัดไม่เกิน 360 นาที (6 ชม.) เพื่อความปลอดภัยและป้องกันสินค้าเสื่อมสภาพ

ความท้าทาย: บีบให้ระบบต้องกระจายงานอย่างสมดุล ไม่สามารถสุมงานให้รถคันเดียวได้

Strategic Insights: 2 แนวคิดพลิกการแก้ปัญหา

INSIGHT 01 : COST-DRIVEN > DISTANCE-DRIVEN

ยอมวิ่งอ้อมขึ้น แต่ลดต้นทุนรวมของธุรกิจได้มากกว่า

อัลกอริทึมทั่วไปมักพยายามลดกิโลเมตร จนเผลอเบิกใช้รถใหญ่ที่มีต้นทุนสูงออกมาวิ่ง
แต่แนวคิดของเราคือ 'รีดประสิทธิภาพรถเล็กให้เต็มที่ก่อนเสมอ' แม้ระยะทางจะเพิ่มขึ้น 5-10% แต่ผลรวมทางการเงินประหยัดกว่า
การนำรถใหญ่ออกมาวิ่งอย่างมหาศาล

INSIGHT 02 : WAITING IS OPTIMIZATION

การจอดรอหน้างาน ไม่ใช่ความสูญเปล่า แต่เป็นกลยุทธ์เวลา

ในมิติเวลา (Time Window) การสั่งให้รถไปถึงก่อนเวลาแล้วจอดรอ (Waiting Slack) ช่วยให้รถสามารถรวมออเดอร์ของหลายร้าน
ในโซนเดียวกันได้ครบถ้วน โดยไม่ต้องสั่งรถอีกคันวิ่งแยกมาเก็บรอบสอง

The Universal VRP Solver Architecture

Decoupled 2 - Tier Architecture: สมองกลตัวเดียว จบทั้งทฤษฎีและโลกจริง

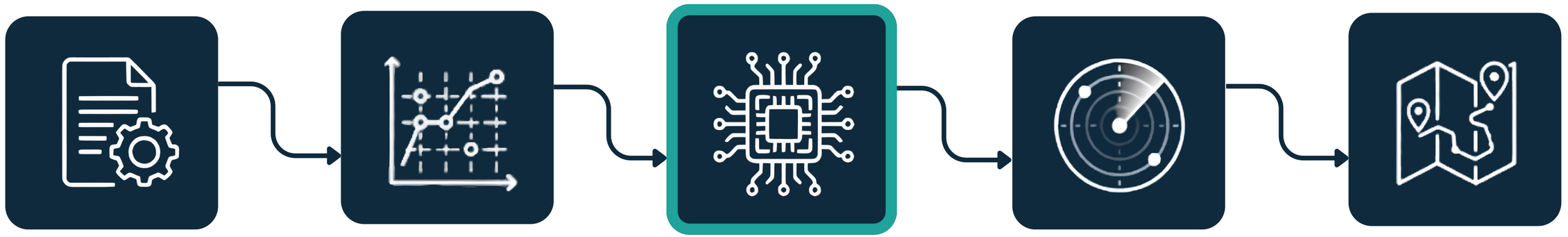
Tier 1 - Academic Benchmark Engine: คำนวณ TSPLIB มาตรฐานสากล พิสูจน์ความแม่นยำด้วย Gap 0.00%

Tier 2 - Business Extension Layer: ปลดล็อกเงื่อนไขโลกจริง (Heterogeneous Fleet + Time Windows + Cost Weights) ผ่านการต่อเติมไฟล์ .json โดยไม่ต้องแก้โค้ดหลัก

The Universal VRP Solver

ผ่านบรรทัดโค้ดเจาะลึก 5 สเตจ: จาก Input Parsing สู่การรัน Metaheuristics แตะผลลัพธ์ Gap 0.00%

The Universal VRP Solver



Input Parsing

อ่านข้อมูลโจทย์และค่าคอนฟิก

Distance Matrix

แปลงพิกัดสู่เมทริกซ์

OR-Tools Model

สร้างมิติข้อจำกัด 3 แกน

Search Engine

ค้นหาและปรับจุดเส้นทาง Metaheuristics

Routes Solution

สรุปผลลัพธ์เส้นทางปฏิบัติจริง

Input Parsing

```python
def parse_vrp_file(filepath: str) -> Dict[str, Any]:
    # 1. สกัด Metadata อัตโนมัติด้วย Regular Expression
    match = re.search(r'CAPACITY\s*:\s*(\d+)', content)
    if match: data['capacity'] = int(match.group(1))

    # 2. สแกนอ่าน Sections สำคัญแบบ Dynamic State-Machine
    for line in content.split('\n'):
        if line.startswith('NODE_COORD_SECTION'): parsing_coords = True
        elif line.startswith('DEMAND_SECTION'):   parsing_demands = True
        elif line.startswith('DEPOT_SECTION'):    parsing_depot = True

        # จัดเก็บพิกัด, ความต้องการสินค้า และจุดโกดัง
        if parsing_coords:   data['coordinates'][node] = (x, y)
        elif parsing_demands: data['demands'][node] = demand
        elif parsing_depot:   data['depot'] = depot_node

    return data
```

Regex Metadata

สกัดข้อมูลความจุรถ (Capacity), จำนวนรถ และ Optimal Value อัตโนมัติจากหัวตารางโดยไม่ต้อง Hardcode

Section Scanner

สแกนแยกบล็อกพิกัด (X, Y), ปริมาณสินค้า (Demand) และจุดโกดัง (Depot) ตามมาตรฐานสากลของ TSPLIB

Dual Data Source

รองรับโครงสร้างไฟล์ .vrp มาตรฐาน และเตรียมพร้อมสำหรับการควบรวมข้อมูลโลกจริงจากไฟล์ .json

Distance Matrix

```python
def compute_euclidean_distance(p1, p2):
    return round(math.sqrt(
        (p1[0] - p2[0])**2 + (p1[1] - p2[1])**2))

# Distance Matrix ของทุกคู่ Node
data['distance_matrix'] = [
    [0 if i == j else compute_euclidean_distance(
        vrp_data['coordinates'][i],
        vrp_data['coordinates'][j])
     for j in sorted_nodes] for i in sorted_nodes]

# ทับค่ามาตรฐานด้วยไฟล์ .json (ถ้ามี)
for key in ['vehicle_capacities', 'vehicle_costs',
            'time_windows', 'service_times',
            'max_time_per_route']:
    if key in cfg: data[key] = cfg[key]
```

Distance Matrix

คำนวณระยะทางเส้นตรงของทุกคู่ Node แล้วปัดเป็นจำนวนเต็มตามมาตรฐาน TSPLIB

ค่ามาตรฐาน

โหมด Standard: ความจุเท่ากันทุกคัน ตัวคูณต้นทุน = 1 ไม่มีเวลา

ทับด้วย .json

ถ้ามีไฟล์ config จะแทนที่ค่ามาตรฐาน (รถ เวลา ต้นทุน)

Model Building & Vehicle Cost

```python
manager = pywrapcp.RoutingIndexManager(
    len(data['distance_matrix']),
    data['num_vehicles'], data['depot'])
routing = pywrapcp.RoutingModel(manager)

# ต้นทุน = ระยะทาง x ตัวคูณของรถแต่ละคัน
for vehicle_id in range(data['num_vehicles']):
    def vehicle_cost_callback(from_index, to_index,
                              v_id=vehicle_id):
        from_node = manager.IndexToNode(from_index)
        to_node = manager.IndexToNode(to_index)
        dist = data['distance_matrix'][from_node][to_node]
        return int(dist * data['vehicle_costs'][v_id])

    cb_index = routing.RegisterTransitCallback(
        vehicle_cost_callback)
    routing.SetArcCostEvaluatorOfVehicle(cb_index, vehicle_id)
```

RoutingIndexManager

แปลงระหว่าง Node ของโจทย์กับ index ภายใน OR-Tools

vehicle_cost_callback

ต้นทุนของแต่ละเส้นทาง = ระยะทาง x ตัวคูณของรถคันนั้น

ผลที่ตามมา

รถใหญ่ (1.8x) แพงกว่า solver จึงต้องชั่งน้ำหนักระหว่างรถเล็กกับรถใหญ่

Vehicle Capacity Constraints

```python
def demand_callback(from_index):
    from_node = manager.IndexToNode(from_index)
    return data['demands'][from_node]

demand_callback_index = routing.RegisterUnaryTransitCallback(
    demand_callback)

routing.AddDimensionWithVehicleCapacity(
    demand_callback_index,
    0,                              # slack
    data['vehicle_capacities'],     # ความจุของรถแต่ละคัน
    True,                           # เริ่มนับที่ 0
    'Capacity')
```

demand_callback

คืนค่าปริมาณพัสดุ (Demand) ของแต่ละ Node

AddDimensionWithVehicleCapacity

สะสม load ตามเส้นทาง และห้ามเกินความจุของรถคันนั้น

ใน FM01

Node 26 มี Demand 19 เกิน 18 ของรถเล็ก จึงต้องไปกับรถใหญ่

Time Windows & Route Duration

```python
def time_callback(from_index, to_index):
    from_node = manager.IndexToNode(from_index)
    to_node = manager.IndexToNode(to_index)
    return (data['service_times'][from_node]
        + data['distance_matrix'][from_node][to_node])

time_callback_index = routing.RegisterTransitCallback(
    time_callback)

routing.AddDimension(
    time_callback_index,
    1000,                          # slack: รอได้
    data['max_time_per_route'],    # เวลาสูงสุดต่อ route
    not has_time_windows,          # False เมื่อมี Time Window
    'Time')

# ช่วงเวลาที่แต่ละ Node เริ่มรับพัสดุได้
time_dimension.CumulVar(index).SetRange(
    time_window[0], time_window[1])
```

time_callback

เวลาจากจุด i ไป j = service time ของ i + ระยะทาง i ถึง j

AddDimension

slack 1000 ทำให้รอได้ และจำกัดเวลารวมต่อ route (360 นาที)

SetRange

เวลาเริ่มรับที่ Node ต้องอยู่ใน [เวลาเปิด, เวลาปิด]

Search Strategy & Metaheuristics

```
search_parameters = pywrapcp.DefaultRoutingSearchParameters()

search_parameters.first_solution_strategy = (
    routing_enums_pb2.FirstSolutionStrategy
    .PATH_CHEAPEST_ARC)

search_parameters.local_search_metaheuristic = (
    routing_enums_pb2.LocalSearchMetaheuristic
    .GUIDED_LOCAL_SEARCH)

search_parameters.time_limit.FromSeconds(time_limit)

solution = routing.SolveWithParameters(search_parameters)

# Gap เทียบ Optimal (เฉพาะโหมด Standard)
gap = ((total_dist - optimal) / optimal) * 100
```

First solution

เริ่มด้วย PATH_CHEAPEST_ARC เพื่อได้คำตอบแรกอย่างรวดเร็ว

Guided Local Search

ปรับปรุงคำตอบต่อเนื่องจนหมด time_limit โดยลงโทษเส้นทางเดิมเพื่อกระโดดข้าม Local Optima

Gap

คิดเฉพาะโหมด Standard ที่ไฟล์ .vrp มีค่า Optimal เพื่อประเมินความแม่นยำเทียบระดับสากล

1 Solver 2 Modes

1 Solver สลับโหมดอัตโนมัติระหว่าง Pure CVRP และ Complex VRPTW

Standard

อ่าน .vrp อย่างเดียว

- ความจุรถเท่ากันทุกคัน
- ไม่มีเวลาจำกัดโดยรวมและในแต่ละจุด
- คำนวณ Gap % เทียบ Optimal

Optional / Real-world

อ่าน .vrp + .json

- รถต่างขนาด + ตัวคูณต้นทุนต่างกัน
- Time Window + Service Time
- เวลา route ไม่เกิน 360 นาที

เริ่มต้น

`PATH_CHEAPEST_ARC`

ปรับปรุง

`GUIDED_LOCAL_SEARCH`

หยุดเมื่อ

`time limit 10–15 วินาที`

Benchmark Performance Validation

A-n32-k5

5 คัน · Cap 100

0.00%

Optimality Gap

Solver **784** Optimal **784**

B-n31-k5

5 คัน · Cap 100

0.00%

Optimality Gap

Solver **672** Optimal **672**

P-n40-k5

5 คัน · Cap 140

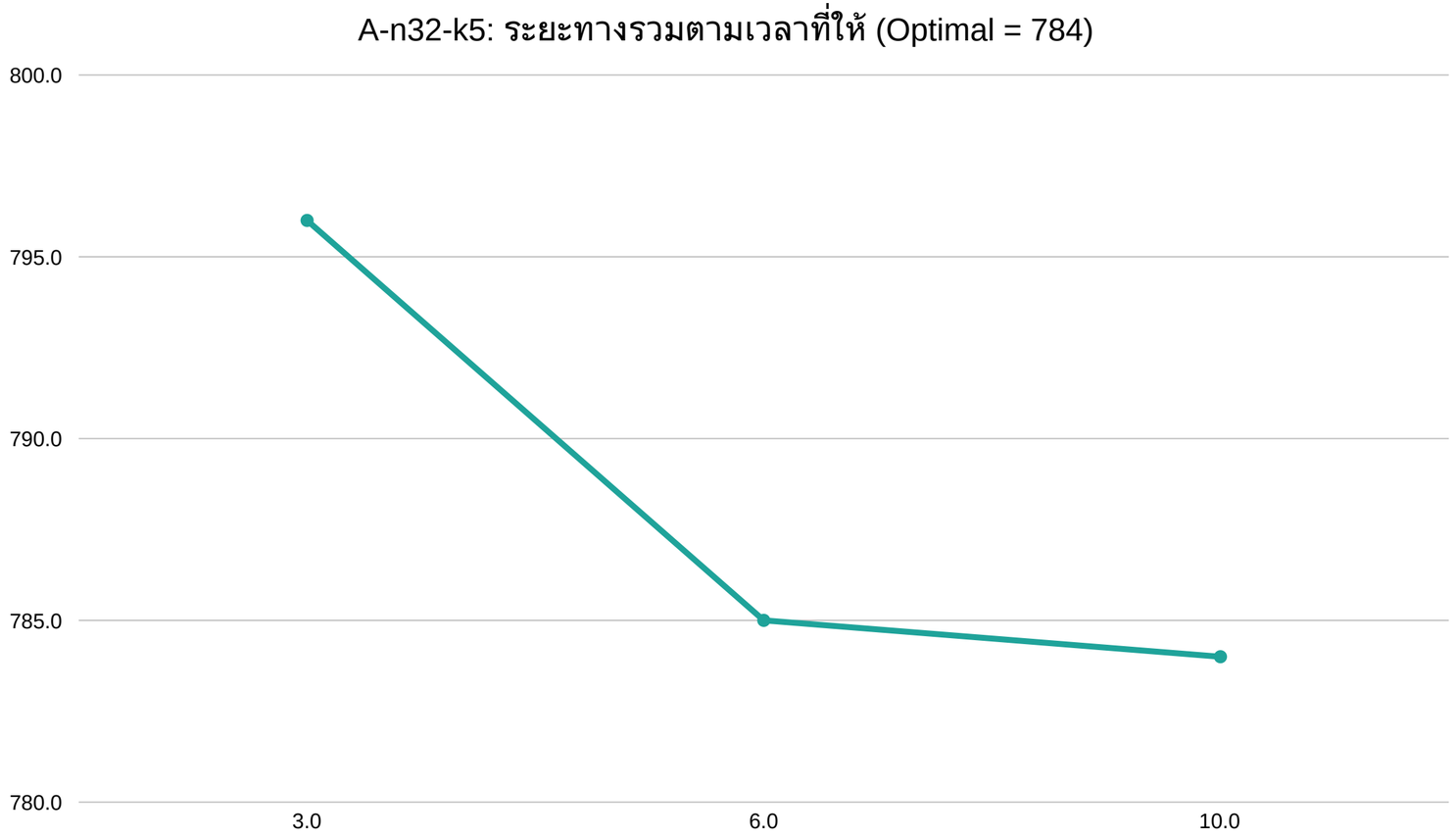
0.00%

Optimality Gap

Solver **458** Optimal **458**

Standard Mode · ระยะทางรวม · Time Limit 10 วินาทีต่อชุด · ทุกชุดได้คำตอบที่ Feasible

ให้เวลาคิดมากขึ้น ผลดีขึ้นไหม



3 Seconds

Gap 1.53%

6 Seconds

Gap 0.13%

10 Seconds

Gap 0.00%

Standard Mode · ทดลองรันซ้ำแยกต่างหาก · ผลขึ้นกับความเร็วในการรัน (รอบที่ 10 วินาทีได้ 784)

Routes Solution: A-n32-k5

(Optimal 784 | Gap 0.00%)

Optimal Convergence (0.00%)

บรรลุ Global Optimum ที่ระยะทางรวม 784 ตรงกับเฉลยมาตรฐานสากล (Optimal: 784) อย่างสมบูรณ์แบบ

Peak Load Efficiency (98%)

จัดลำดับส่งสินค้าได้แน่นเต็มพิกัด โดยรถคันที่ 0, 1, 3 บรรทุกสูงถึง 98/100 หน่วย (98.0%) คุ้มค่าสูงสุด

Zero Over-capacity & Feasible

ส่งครบทั้ง 31 ลูกค้า (Demand 410) ไม่มีคันใดบรรทุกเกิน 100 และทุกลำดับวนกลับเข้าโกดัง (Depot 1) ครบถ้วน

```
[Vehicle 0] (Capacity: 100 | Cost Multiplier: 1x)
  Route : Depot 1 -> Node 7 -> Node 3 -> Node 4 -> Node 24 -> Node 5
          -> Node 12 -> Node 29 -> Node 15 -> Depot 1
  Stats : Load: 98/100 (98.0%) | Distance: 230 | Cost: 230.0

[Vehicle 1] (Capacity: 100 | Cost Multiplier: 1x)
  Route : Depot 1 -> Node 21 -> Node 6 -> Node 26 -> Node 11 -> Node 16
          -> Node 23 -> Node 10 -> Node 9 -> Node 19 -> Node 30 -> Depot 1
  Stats : Load: 98/100 (98.0%) | Distance: 267 | Cost: 267.0

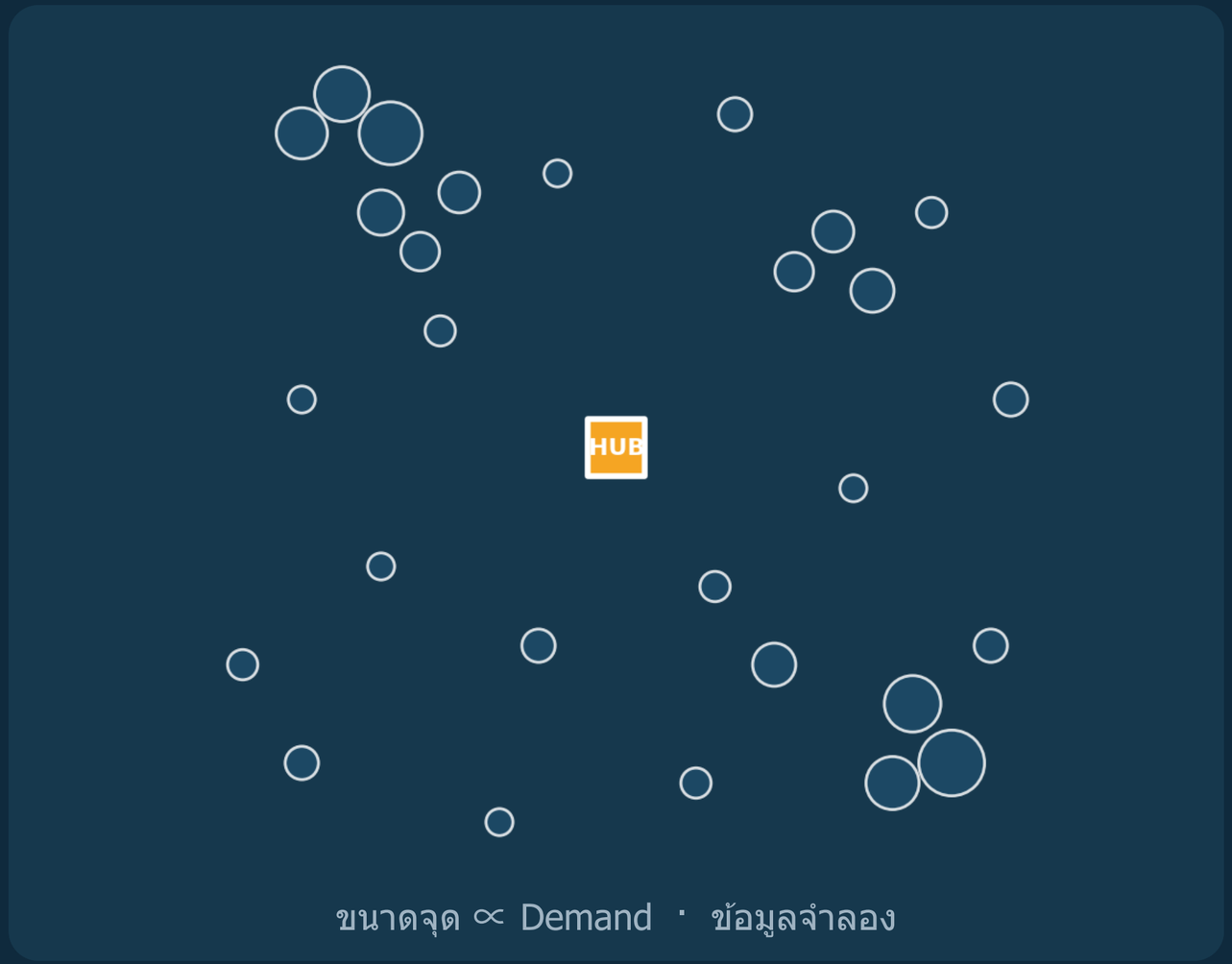
[Vehicle 2] (Capacity: 100 | Cost Multiplier: 1x)
  Route : Depot 1 -> Node 13 -> Node 2 -> Node 17 -> Node 31 -> Depot 1
  Stats : Load: 72/100 (72.0%) | Distance: 73 | Cost: 73.0

[Vehicle 3] (Capacity: 100 | Cost Multiplier: 1x)
  Route : Depot 1 -> Node 22 -> Node 32 -> Node 20 -> Node 18
          -> Node 14 -> Node 8 -> Node 27 -> Depot 1
  Stats : Load: 98/100 (98.0%) | Distance: 155 | Cost: 155.0

[Vehicle 4] (Capacity: 100 | Cost Multiplier: 1x)
  Route : Depot 1 -> Node 25 -> Node 28 -> Depot 1
  Stats : Load: 44/100 (44.0%) | Distance: 59 | Cost: 59.0

================================================================================
  Status: SUCCESS | Solve Time: 10.01s | Vehicles Utilized: 5 / 5
  Total Distance: 784 | Optimal Target: 784 | Optimality Gap: 0.00%
```

Self-created case: FM01



FM01-n29-k6

- จำนวน Node ทั้งหมด = 29 Nodes
- Depot = 1 จุด
- Seller = 28 จุด
- จำนวนรถ = 6 คัน
- Demand รวม = 190 หน่วย
- Total Capacity = 232 หน่วย
- Capacity Utilization = 81.9%

รอบเย็น 16:00–22:00

แต่ละ seller เปิดรับไม่พร้อมกัน

โจทย์ที่ต้องตอบ

seller ไหน → รถคันไหน → ลำดับใด

FM01: ออกแบบข้อมูลและข้อจำกัด

29

Nodes (Hub + 28)

190

Demand รวม

6

รถ (2 ประเภท)

360

นาที / route



รถเล็ก ×4

Cap 18

Cost 1.0×



รถใหญ่ ×2

Cap 80

Cost 1.8×



Service Time

5–37 min



3 Seller Types

ร้านค้า ธุรกิจรายย่อย

SME ธุรกิจขนาดกลาง

Warehouse ธุรกิจขนาดใหญ่

ข้อจำกัดที่ solver ต้องทำพร้อมกัน

FM01: ผลลัพธ์ Final Route

Real-world Mode · Heterogeneous Fleet · Time Window · Service Time · Capacity

```
[Vehicle 0] (Capacity: 18 | Cost Multiplier: 1.0x)
  Route : Node 1 [Time: 0] -> Node 3 [Time: 90] -> Node 4 [Time: 118]
          -> Node 5 [Time: 142] -> Node 17 [Time: 170] -> Depot 1 [Time: 208]
  Stats : Load: 15/18 (83.3%) | Distance: 103 | Cost: 103.0

[Vehicle 2] (Capacity: 18 | Cost Multiplier: 1.0x)
  Route : Node 1 [Time: 0] -> Node 2 [Time: 32] -> Node 11 [Time: 56]
          -> Node 12 [Time: 78] -> Node 13 [Time: 98] -> Node 14 [Time: 130]
          -> Node 10 [Time: 153] -> Depot 1 [Time: 186]
  Stats : Load: 17/18 (94.4%) | Distance: 141 | Cost: 141.0

[Vehicle 3] (Capacity: 18 | Cost Multiplier: 1.0x)
  Route : Node 1 [Time: 0] -> Node 8 [Time: 24] -> Node 23 [Time: 49]
          -> Node 15 [Time: 82] -> Node 9 [Time: 110] -> Depot 1 [Time: 135]
  Stats : Load: 16/18 (88.9%) | Distance: 95 | Cost: 95.0

[Vehicle 4] (Capacity: 80 | Cost Multiplier: 1.8x)
  Route : Node 1 [Time: 0] -> Node 20 [Time: 60] -> Node 21 [Time: 79]
          -> Node 24 [Time: 112] -> Node 25 [Time: 145] -> Node 26 [Time: 182]
          -> Node 22 [Time: 226] -> Depot 1 [Time: 273]
  Stats : Load: 67/80 (83.8%) | Distance: 97 | Cost: 174.6

[Vehicle 5] (Capacity: 80 | Cost Multiplier: 1.8x)
  Route : Node 1 [Time: 0] -> Node 18 [Time: 31] -> Node 19 [Time: 54]
          -> Node 6 [Time: 83] -> Node 7 [Time: 112] -> Node 16 [Time: 148]
          -> Node 27 [Time: 176] -> Node 28 [Time: 211] -> Node 29 [Time: 255]
          -> Depot 1 [Time: 328]
  Stats : Load: 75/80 (93.8%) | Distance: 164 | Cost: 295.2

================================================================================
  Status: SUCCESS | Solve Time: 10.04s | Vehicles Utilized: 5 / 6
  Total Distance: 600 | Total Objective Cost: 808.8
```

ผลลัพธ์จาก Solver

5 / 6 Vehicles Used	**600** Total Distance
808.8 Objective Cost	**328 min** Longest Route

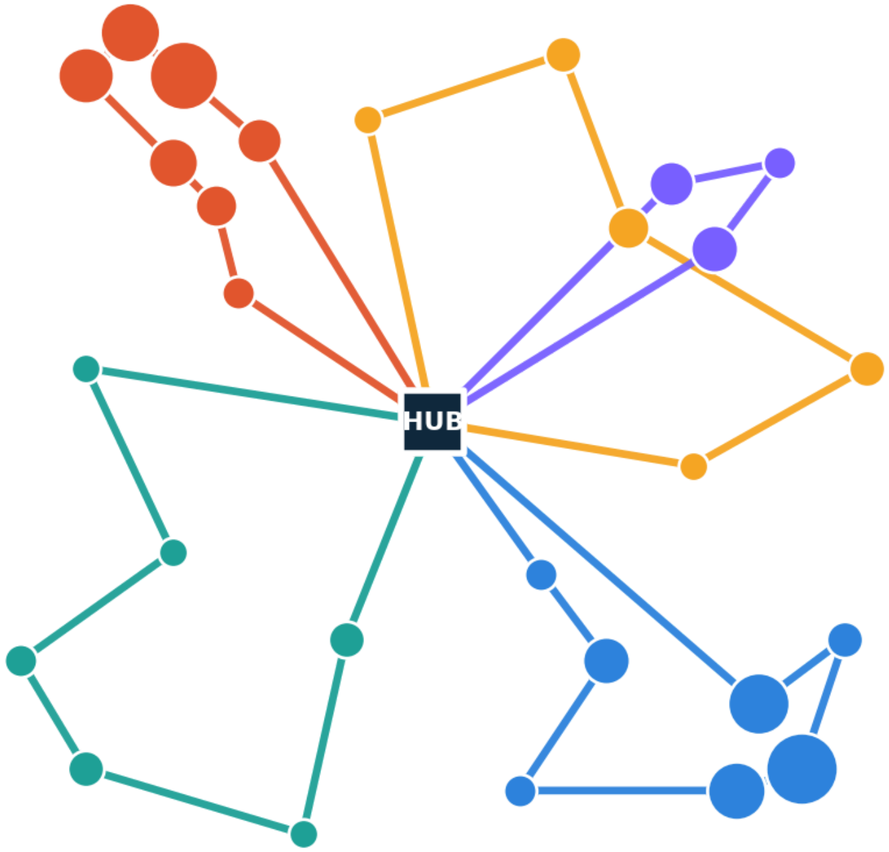
✓ ผ่านทุก Constraints

Seller 28 จุดครบ · Capacity ผ่าน · Time Window ผ่าน

Route ช้าที่สุดจบ 21:28

ยังเหลือ Buffer 32 นาที ก่อน 22:00

FM01: ผลลัพธ์ของ Solver



600	808.8	5 / 6
Total Distance	Objective Cost	รถที่ใช้

	Load / Capacity		Utilization	ระยะทาง
V0 เล็ก	15/18		83.3%	103
V2 เล็ก	17/18		94.4%	141
V3 เล็ก	16/18		88.9%	95
V4 ใหญ่	67/80		83.8%	97
V5 ใหญ่	75/80		93.8%	164

รถ V1 รถเล็ก ไม่ถูกใช้ · Best Feasible Solution Found

FM01: สิ่งที่ค้นพบ

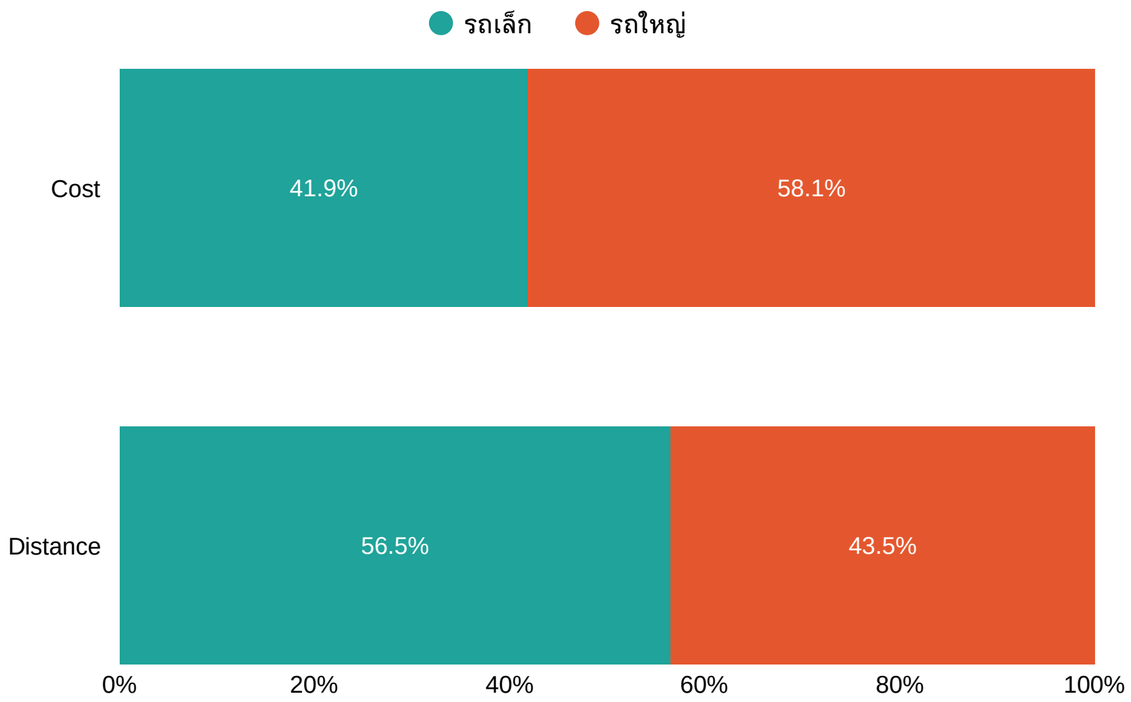
Node 26 ต้องใช้รถใหญ่

“Node 26 ต้องใช้รถใหญ่”
Demand 19 > Small Vehicle Capacity 18

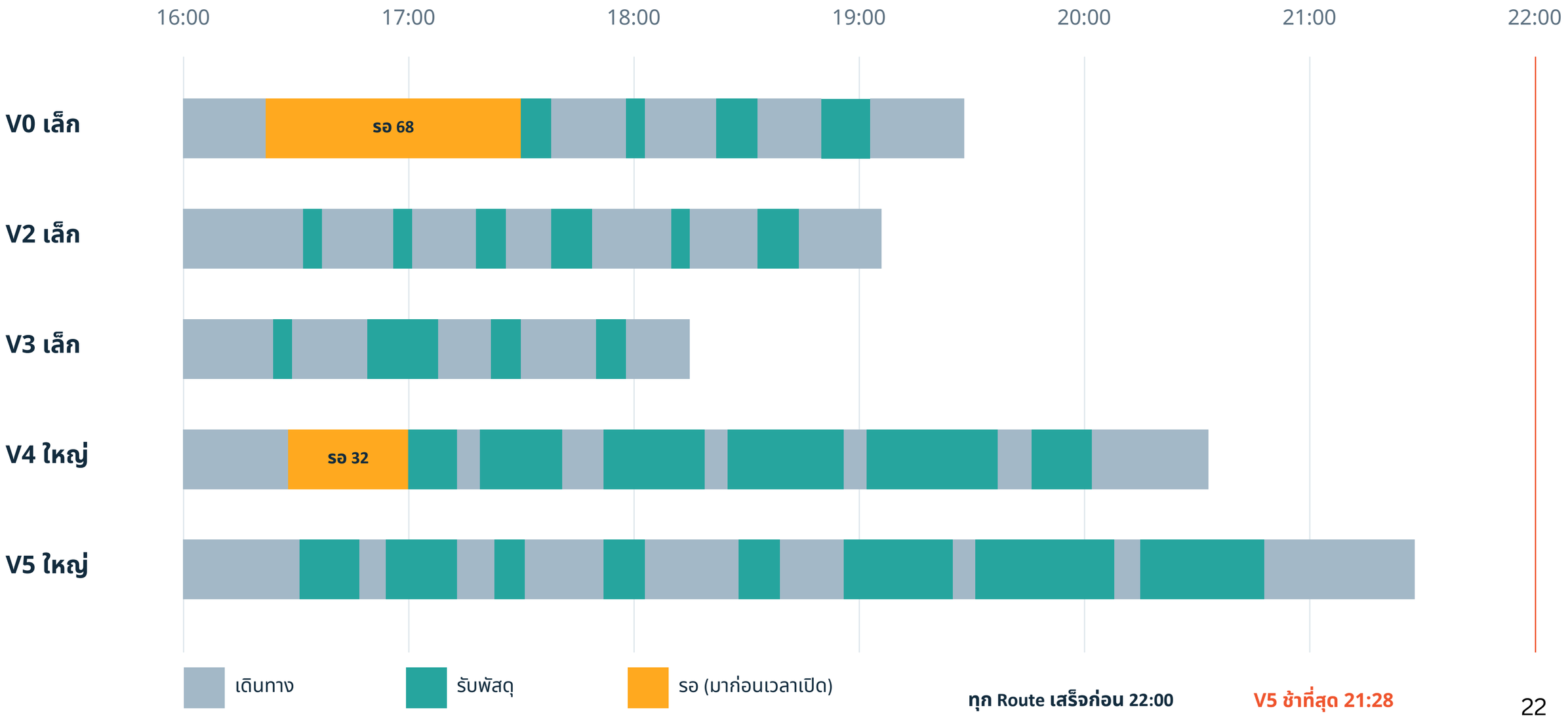
“Time Window → รอได้เมื่อมาก่อนเวลา

Node 3: ช่วง 90–135 เริ่มที่นาที 90
Node 28: ช่วง 180–300 เริ่มนาทีที่ 211

สัดส่วนรถใหญ่ 2 คัน: ใช้ระยะทางน้อยกว่า แต่แพงกว่า



Time Windows ส่งผลต่อแผนการเดินรถ



Insight: Time Window ทำให้เกิด Waiting (V0 รอ 68 นาที, V4 รอ 32 นาที) แต่แผนยังอยู่ในกรอบเวลา 360 นาที

เหลือ Buffer 32 นาที

ข้อจำกัด และต่อยอด

ข้อจำกัดของเคสนี้

- พิกัด 2D · Euclidean ไม่สะท้อนโครงข่ายถนนจริง
- Travel Time ไม่เปลี่ยนตามสภาพการจราจร
- Demand รู้ล่วงหน้า ไม่มีเหตุไม่คาดฝัน
- ตัวคูณต้นทุนเป็นค่าจำลอง

ต่อยอดได้

- ใช้ระยะทางถนนจริง (Maps / OSRM)
- เพิ่ม Time-dependent Traffic
- Dynamic Re-routing เมื่อ Demand / Order เปลี่ยน
- เพิ่ม Fixed Cost ต่อการเปิดใช้รถแต่ละคัน

ผลของ FM01 คือ Best Feasible Solution Found ยังไม่ได้พิสูจน์ว่าเป็น Global Optimum

Thank You



Google OR-Tools · Python